

Automated Blast Radius Measurement and Reduction in AWS Cloud Workloads Using Attack Graph Analysis

Name: Soumik Shadman (11154499)

Program: M.Sc. Communication Systems and Networks

Institution: Technische Hochschule Köln (TH Köln)

Company: Dyn Media GmbH (Axel Springer SE)

1. Problem Statement

Cloud workloads on AWS operate under an implicit trust model where compute resources can reach far more services and data than they functionally require. If an attacker compromises a single component, the resulting blast radius — the set of resources the attacker can access — is unknown and unnecessarily large.

Existing AWS services (Security Hub, Inspector, IAM Access Analyzer) identify individual misconfigurations but do not provide a unified, quantitative measure of blast radius from a specific compromised resource. Open-source tools (Prowler, ScoutSuite) perform compliance checks but do not model inter-resource reachability or rank controls by their actual impact on reducing lateral movement.

No established method exists to: 1. Automatically measure how far an attacker can reach from a compromised resource 2. Quantify the effectiveness of individual security controls on reducing that reach 3. Validate automated findings against a formal mathematical model

2. Research Objectives

1. Define a formal blast radius model for AWS workloads based on graph theory
 2. Develop an automated scanner that discovers resources, maps reachability, and calculates the blast radius score
 3. Evaluate the effectiveness of security controls by applying hardening measures progressively and measuring the reduction
 4. Validate the model by comparing mathematical predictions against scanner results
 5. Demonstrate generalizability by applying the method to two workloads with different architectures
-

3. Research Questions

RQ1: How can blast radius in AWS cloud workloads be formally modeled and automatically measured?

RQ2: Which security controls produce the greatest measurable reduction in blast radius?

RQ3: Does control effectiveness differ between traditional (EC2+RDS) and serverless (Lambda+DynamoDB) architectures?

RQ4: How accurately does the mathematical model predict the blast radius measured by the automated scanner?

4. Blast Radius Model

The model represents an AWS account as a directed graph:

$G = (V, E)$

- **V** = set of AWS resources (compute instances, databases, storage, endpoints, external routes)
- **E** = set of directed edges representing confirmed reachability

An edge $(u, v) \in E$ exists if at least one of the following holds: - **Network reachability:** Security group rules permit traffic from u to v and a network route exists - **IAM reachability:** The IAM role attached to u has permission to perform actions on v - **Metadata exploitation:** u runs on an instance with IMDSv1 enabled, permitting credential theft

Blast Radius Score:

$$BR(v) = |R(v)| / |V \setminus \{v\}|$$

Where: - v = the compromised entry point - $R(v)$ = the set of all nodes reachable from v via directed paths (BFS traversal) - $|V \setminus \{v\}|$ = total number of other resources in the account

$BR(v) = 0$ indicates full containment. $BR(v) = 1$ indicates full reachability.

Control Effectiveness:

$$CE(c) = BR_before(v) - BR_after(v)$$

Where c is a specific security control applied. Higher CE indicates greater impact.

Origin: The attack graph formalism $G=(V,E)$ is established by Ou et al. (2006) and adopted by Capobianco et al. (2019). The BFS traversal method for reachability is validated by Ibrahim et al. (2019) for microservice architectures. The blast radius scoring formula and control effectiveness metric are contributions of this thesis.

5. Automated Scanner

A Python-based tool that implements the model against live AWS infrastructure:

1. **Discovery** — Connects to AWS APIs to enumerate all resources in a target account (compute, databases, storage, networking, endpoints)
2. **Reachability Mapping** — Analyzes security group rules, route tables, IAM role policies, and IMDS configuration to determine which paths exist
3. **Graph Construction** — Builds a directed graph where resources are nodes and confirmed paths are edges, categorized by type (network, IAM, metadata, route)
4. **Scoring** — Performs BFS from the entry point, counts reachable nodes, computes the blast radius percentage, and calculates per-edge impact by temporarily removing each edge and recalculating

Technology: Python 3.11, boto3 (AWS SDK), networkx (graph library). The tool is read-only — it observes configurations without modifying any resources.

6. How the Scanner Produces the Score (Example)

Given an AWS account with a WordPress workload, the scanner operates as follows:

Step 1 — Discovery finds these resources (nodes):

#	Resource	Type
1	EC2 WordPress instance	Compute (entry point)
2	RDS MySQL database	Database
3	S3 media bucket	Storage
4	S3 backup bucket	Storage
5	SSM VPC endpoint	Endpoint
6	Internet (via NAT Gateway)	External

Total nodes: 6. Entry point = EC2. Other nodes = 5.

Step 2 — Reachability mapping checks paths such as:

- Can the EC2 instance connect to the database? (security group rules + network route)
- Can it reach the internet? (route table entries + egress rules)
- Can credentials be stolen via metadata? (IMDS configuration)
- What AWS services can its IAM role access? (attached policies)

Each confirmed path becomes an edge in the graph.

Reachable from EC2 (hypothetical): all 5 other resources = **5 nodes**

Step 3 — Scoring:

$$BR(EC2) = |R(EC2)| / |V \setminus \{EC2\}| = 5 / 5 = 100\%$$

If the attacker can reach all resources, the blast radius is 100%.

Step 4 — Edge impact analysis (expected):

The scanner removes each edge one at a time and recalculates. We expect results such as:

If We Remove...	Estimated Impact
Internet egress path	Likely the highest reduction (~40-60%) since multiple resources may only be reachable through the internet route
Database access from broad sources	Moderate reduction (~10-20%)
Overly permissive endpoint policies	Lower reduction (~5-10%)

The actual values will be determined by running the scanner against the live environment during the research phase.

6.1 Why Multiple Analysis Dimensions Are Required

No single analysis method captures the full blast radius. Each dimension reveals paths that others miss:

Analysis Dimension	What It Checks	Example Edge It Finds
Network (SGs + Routes)	Can traffic flow between resources?	EC2 can connect to RDS on port 3306
IAM (Role Policies)	What AWS API calls can the resource make?	EC2's role can read S3 buckets
Metadata (IMDS)	Can credentials be stolen from the instance?	IMDSv1 allows SSRF to extract IAM session tokens

If we only analyze one dimension, we undercount the reachable resources:

Network analysis alone → finds some edges → partial blast radius
+ IAM analysis → finds more edges → blast radius increases
+ Metadata analysis → finds more edges → full blast radius revealed

Hypothetical example:

Dimensions Included	Edges Found	Estimated Reachable Resources
Network only	2–3	~30-40% of resources
Network + IAM	4–6	~50-70% of resources
Network + IAM + Metadata	5–8	~70-100% of resources

This demonstrates why existing tools that check only one dimension (e.g., Inspector checks only network, IAM Access Analyzer checks only IAM) underestimate the true blast radius. The combined model captures attack paths that span multiple dimensions — for example, an attacker uses a metadata vulnerability to steal credentials (metadata dimension), then uses those credentials to access an S3 bucket (IAM dimension) that is not network-reachable from the original instance.

The scanner builds one unified graph combining all dimensions, then traverses it to produce the complete blast radius score.

7. Security Controls Under Evaluation

Primary Controls: - IMDSv2 enforcement — blocks credential theft via server-side request forgery - Database security group restriction — limits database access to authorized compute only

Secondary Controls: - Internet egress removal — eliminates data exfiltration paths - VPC endpoint policy scoping — restricts API access to approved resources

Controls are applied one at a time. The scanner runs after each to produce a progressive reduction curve.

8. Dual Validation

The blast radius is calculated two ways:

- **Mathematical prediction** — theoretically computed from the architecture configuration using the graph model
- **Automated measurement** — empirically measured by the scanner against the live account

Discrepancies between prediction and measurement are analyzed to identify limitations of each method.

9. Target Environments

Workload	Architecture	Security Model
WordPress	EC2 + RDS + S3	Network-centric (security groups, route tables)
Article Generator	Lambda + DynamoDB + S3	Identity-centric (IAM roles, service trust boundaries)

Both are production systems at Dyn Media GmbH, deployed via AWS CDK.

10. Timeline

Weeks	Activity
1–2	Formal model definition, literature review
3–5	Scanner development (discovery, reachability, graph, scoring)
6	Baseline measurements on target accounts
7–8	Progressive hardening and measurement
9	Cross-architecture comparison
10	Model validation and discrepancy analysis
11–12	Thesis writing and defense preparation

11. Expected Contributions

1. A formalized blast radius metric for AWS cloud workloads
2. An automated measurement tool implementing the metric
3. Empirical evaluation of control effectiveness, ranked by impact
4. Validation comparing mathematical predictions against automated measurements
5. Cross-architecture analysis demonstrating generalizability

12. References

- Capobianco, F. et al. “Employing Attack Graphs for Intrusion Detection.” NSPW ’19, ACM, 2019.
Formal attack graph model $G=(V,E)$; proactive graph generation from configurations.
- Ibrahim, A. et al. “Attack Graph Generation for Microservice Architecture.” SAC ’19, ACM, 2019.
BFS-based attack graph traversal for containerized systems; scalability validation.

- Stournaras, A. “HackerGraph: Creating a Knowledge Graph for Security Assessment of AWS Systems.” KTH Royal Institute of Technology, 2023. *Knowledge graph for AWS security; IAM privilege escalation modeling. Tested on lab environments only.*
- Ou, X. et al. “A Scalable Approach to Attack Graph Generation.” ACM CCS, 2006.
- NIST SP 800-207. “Zero Trust Architecture.” 2020.
- AWS Well-Architected Framework — Security Pillar. 2024.
- Prowler — Open-source security assessment tool. <https://github.com/prowler-cloud/prowler>
- ScoutSuite — Multi-cloud security auditing tool. <https://github.com/nccgroup/ScoutSuite>